

Concurrencia

Material de lectura sobre el tema: Complementar lo presentado aquí con:

- Sebesta: Capítulo 13
- Scott: Sección 2.3 del capítulo 13, “*Thread Creation Syntax*”.

En general, un programa se dice concurrente si puede tener más de un contexto de ejecución activo. La idea subyacente es conseguir implementaciones que puedan capturar la estructura lógica del problema aprovechando la disponibilidad de procesadores para maximizar la velocidad de cómputo.

En contexto donde el paralelismo no es posible, por la falta de recursos, la concurrencia lógica entra en juego. En este segundo caso se caracteriza a sistemas donde dos o más tareas están en ejecución “al mismo tiempo” (aunque, desde el punto de vista de la ejecución real, lo que está sucediendo es que se intercalan los puntos de ejecución pero de manera no predecible).

A la hora de programar de forma concurrente, todo programador debe ser consciente de los niveles de detalles de esta facilidad y cómo sacar provecho de ella. Para ello el lenguaje de programación debe contar con habilidades para expresar concurrencia.

El lenguaje de programación puede concebirse como un lenguaje concurrente desde su diseño o en caso de carecer de facilidades para la programación concurrente puede obtener esas facilidades de dos formas: extendiendo el lenguaje con construcciones adicionales; o bien, por medio de librerías que hagan uso de las facilidades que provea el sistema operativo para dar soporte a la ejecución concurrente. La forma como el lenguaje plantea su vínculo con este concepto determina diferentes niveles de abstracción:

- **Modo “caja negra”:** Este modo es el más abstracto. La concurrencia se incorpora al lenguaje de través de librerías. Es un modo de incorporación bastante sencillo para el lenguaje, pero que en general no permite aprovechar las facilidades de la concurrencia a un nivel óptimo.
- **Modo intermedio:** Requiere que el lenguaje cuente con constructores dedicados para definir tareas y habilidades de sincronización. El programador debe ser consciente de que ciertas tareas deben ser mutuamente independientes.
- **Modo experto:** El lenguaje provee mecanismos para trabajar con concurrencia a bajo nivel. De esta manera, el programador puede necesitar de una comprensión detallada de hardware y sistemas de run-time para implementar los mecanismos de sincronización de más alto nivel.

Aspectos fundamentales

En líneas generales la concurrencia implica la presencia de tres elementos en ejecución:

- Un *hilo* que ejecuta secuencialmente sentencias de un lenguaje de programación. La función de este hilo es ir modificando estados de ejecución.
- El *estado de ejecución* consiste en los elementos de datos creados por un objeto, incluidos sus datos locales, bloque local y activaciones de unidad, además de la ubicación de ejecución actual. Un lenguaje de programación determina lo que constituye un estado de ejecución y, por lo tanto, el estado de ejecución es una propiedad elemental de la semántica. Se produce un cambio de contexto cuando un subproceso o unidad cambia de un estado de ejecución a otro.
- La *exclusión mutua* es el mecanismo que otorga a un subproceso acceso exclusivo a un recurso durante un período de tiempo.

Las dos primeras propiedades representan el mínimo necesario para realizar la ejecución, son tan fundamentales que no se pueden expresar de manera independiente de la máquina o del lenguaje. Por ejemplo, la creación de un nuevo hilo requiere la creación de información de control en tiempo de ejecución del sistema, la manipulación de los estados de ejecución requiere operaciones específicas de la máquina (modificar los punteros de la pila y el registro de activación). Por su parte, la exclusión mutua puede expresarse de forma algorítmica (por ejemplo, implementando el algoritmo de Dekker), sin embargo esta forma de expresarla es propensa a errores y computacionalmente muy costosa, es por eso que en la literatura se propone que la exclusión mutua sea una propiedad de ejecución elemental. Por lo tanto, **cualquier lenguaje de programación que admita concurrencia debe proporcionar construcciones primitivas para implementar estas propiedades.**

Es así como concluimos que, independientemente de la estrategia que tome el lenguaje para brindar soporte para concurrencia, debe brindar facilidades para identificar qué subprogramas o unidades son concurrentes, e indicar formas para que puedan comunicarse y sincronizarse.

A la hora de agregar concurrencia en un lenguaje de programación también resulta relevante analizar el tipado. No es lo mismo trabajar con un lenguaje con tipado estático que con uno con tipado dinámico. Los lenguajes con tipado estático tienen requisitos adicionales para el mecanismo de comunicación. Otro aspecto a tener en cuenta es el grado de flexibilidad que brinda el lenguaje con respecto a los lugares donde se pueden declarar las tareas. Por ejemplo, si las tareas se parecen a los objetos, entonces debería ser posible declarar una tarea en cualquier lugar donde se pueda declarar un objeto.

Tareas, procesos, hilos

En la bibliografía de lenguajes de programación estos términos no son utilizados de manera consistente. Algunos autores los utilizan con semánticas diferentes.

Podemos definir a la **tarea** como una unidad que puede ser ejecutada en forma concurrente con otras unidades. ¿Se parece una tarea a otras construcciones del lenguaje? Claramente hay un vínculo con el concepto de unidad y también con TDA. Un lenguaje de programación no orientado a objetos podría presentar una tarea como una unidad de ejecución independiente, análogo al cuerpo de una unidad no concurrente. Un lenguaje de programación orientado a objetos puede presentar una tarea como un objeto, con una interfaz definida por un conjunto de funcionalidades. La definición deja de manifiesto que de alguna manera es necesario diferenciar a una tarea de una unidad ordinaria. Esta diferenciación es importante porque el comportamiento no es el mismo y por lo tanto la semántica tampoco lo es.

- Una tarea puede ser comenzada en forma implícita.
- El comienzo de la ejecución de una tarea no implica la suspensión del llamador.
- Al completarse la ejecución de una tarea, el control puede o no retornar a la unidad que comenzó su ejecución.

En este punto, los lenguajes de programación deben considerar cómo incorporar concurrencia a nivel unidad o bloque de sentencias (constructores de tarea). Existen dos categorías a considerar:

- **Heavyweight task:** Son tareas que cuentan con su propio espacio de direcciones y su propia pila de ejecución.
- **Lightweight task:** En este caso todas las tareas se ejecutan sobre el mismo espacio de nombre y comparten la pila de ejecución.

Comunicación

La comunicación entre tareas es fundamental ya que establece la posibilidad de conseguir información producida por otra tarea o hilo. El lenguaje de programación deberá proveer algún mecanismo para esto, siendo los más frecuentes los mecanismos de pasaje de mensajes y memoria compartida.

En el caso de adoptar pasaje de mensajes, las tareas o hilos no tienen un estado común, es por eso que para comunicarse se debe realizar un pedido explícito para transmitir datos. En el caso de la memoria compartida, las variables que son accesibles para varias tareas o hilos se encuentran en un área de memoria compartida.

La comunicación de tareas por medio de pasaje de mensajes podría parecerse a una llamada de unidad, con datos pasados como argumentos y recibidos como parámetros, o las tareas podrían comunicarse a través de “canales” intermedios que serían entidades funcionalmente similares a un archivo de entrada/salida. La comunicación podría involucrar nuevas operaciones para enviar y recibir “mensajes” (entidades u objetos de datos) que los programas deben procesar.

Otro aspecto a tener en cuenta en el diseño de lenguajes vinculado a la comunicación es si la misma es directa entre tareas o si se requiere de un intermediario en la comunicación (por ejemplo, una estructura de datos tipo monitor). En caso de requerir de un intermediario, la velocidad de ejecución se puede ver disminuida a causa de la sincronización y transferencia de datos adicional entre las tareas y el intermediario.

Tanto la comunicación sincrónica como la asincrónica son necesarias en un sistema concurrente. En la comunicación sincrónica, una tarea que transmite información suspende la ejecución hasta que otra tarea la recibe y responde; en la comunicación asincrónica, el transmisor puede continuar la ejecución antes de que el receptor reciba los datos. Como la comunicación asincrónica puede implementarse a partir de la sincrónica y viceversa, los lenguajes que adopten pasaje de mensajes como forma de comunicación sólo necesitan proporcionar uno de los dos mecanismos.

Contar con múltiples formas de comunicación en un lenguaje de programación puede resultar confuso para los usuarios o programadores. Además, pueden resultar en pérdidas de eficiencia al requerir de más transformaciones de los datos durante la comunicación.

Sincronización

Por otra parte, la sincronización permite controlar el orden relativo de operaciones pertenecientes a diferentes tareas o hilos. Resulta fundamental para lograr la correctitud lógica de algunas implementaciones concurrentes. Hay dos tipos básicos de sincronización: *cooperativa* y *competitiva (o de exclusión mutua)*. En el primer caso, una tarea debe esperar que otra complete una actividad específica para retomar su ejecución; en el segundo caso, más de una tarea debe hacer uso del mismo recurso, pero por correctitud no puede ser utilizado simultáneamente.

Al momento de resolver cómo incorporar los dos tipos de sincronización, debe establecerse si son implícitas y limitadas en el alcance textual, o son explícitas y están ligadas al flujo de control. De la evaluación de diferentes lenguajes puede notarse que exigirle al programador que cree soluciones con exclusión mutua complejas, como monitores, a partir de primitivas de exclusión mutua de bajo nivel, como semáforos, termina llevando a soluciones incorrectas. Contar con herramientas que permitan reducir la porción de código afectada a la sincronización reduce los errores en las unidades concurrentes.